Strategy Framework & Trading Strategies

Zhuo Zhao joezhao@bu.edu

C++ HFT Study Group | Week 7 | Chapters 9 & 10

Building Low-Latency Applications with C++

Positions & PnL Tracking • Signal Generation • Order Management
Risk Controls • Market Making • Liquidity Taking • End-to-End Integration

The Big Picture: What Does a Trading Strategy Need?

An automated trading system runs a continuous loop:

```
Market data arrives -> Update internal state -> Make trading decision -> Send order
-> Receive confirmation -> Update state -> ...
```

Around this loop, the book decomposes the Trading Engine into five components:

1. FeatureEngine — Computes numerical signals from raw market data (fair price, trade pressure).

2. PositionKeeper — Tracks current position, realized PnL, unrealized PnL, and total PnL.

3. OrderManager — Manages live orders: sending new orders, cancelling stale ones, tracking state.

4. RiskManager — Pre-trade gatekeeper: checks order size, position limits, and loss limits before every new order.

5. Strategy (MarketMaker or LiquidityTaker) — The decision layer that calls the above components.

All five components run on a **single thread** (the TradeEngine thread). Communication with external threads (MarketDataConsumer, OrderGateway) happens exclusively through **lock-free SPSC queues**. There are no mutexes anywhere in the system.

Components inside the TradeEngine are connected via callbacks (std::function). For example, when the order book updates, it notifies FeatureEngine and the strategy through callbacks. This decouples the framework from specific strategy implementations.

00 — Side Enum & Helper Functions (Prerequisite)

Before diving into the components, we need to understand two helper functions that appear everywhere in the codebase. The Side enum is defined as:

```
enum class Side : int8_t { INVALID=0, BUY=1, SELL=-1, MAX=2 };
```

BUY=+1 and SELL=-1 serve a dual purpose. Two separate functions extract the right meaning:

sideToValue() — For Arithmetic

```
inline constexpr auto sideToValue(Side side) noexcept {
    return static_cast<int>(side);  // BUY→+1, SELL→-1
}
```

Returns the raw enum value. Used for branchless position updates: position_ += qty * sideToValue(side). BUY adds, SELL subtracts, no if/else needed.

sideToIndex() — For Array Subscripts (Unsigned Overflow Trick)

```
inline constexpr auto sideToIndex(Side side) noexcept {
    return static_cast<size_t>(side) + 1;
}
```

SELL=-1 cannot be an array index. The trick: static_cast<size_t>(-1) gives SIZE_MAX (unsigned wraparound, defined behavior in C++), then +1 wraps back to 0. So SELL→0, BUY→2. Arrays sized with sideToIndex(Side::MAX)+1 = 4 use indices 0 and 2, wasting two slots (16 bytes) for completely branchless indexing. This pattern appears in open_vwap_, OMOrderSideHashMap, and throughout the system.

01 — FeatureEngine (Signal Generation)

FeatureEngine has a single responsibility: extract numerical signals from raw market data so that strategies don't need to look at raw order book data directly.

The book implements just two features — one for each strategy type. Both are intentionally simple; the book repeatedly emphasizes that there is no single correct formulation.

Feature 1: mkt_price_ (Fair Market Price)

```
mkt_price_ = (bid_price * ask_qty + ask_price * bid_qty) / (bid_qty + ask_qty)
```

This is a quantity-weighted mid price. The intuition: if there's much more quantity on the bid side than the ask side, it signals strong buying pressure. The fair price gets pulled toward the ask (higher). Conversely, if the ask side has more depth, the fair price gets pulled toward the bid.

Note that the weights are the “opposite” side’s quantity: bid_price is multiplied by ask_qty, not bid_qty. Why? Because the quantity sitting on one side represents the “resistance” to price moving in that direction. Heavy bid-side depth means strong buying demand, which pushes the fair price upward (toward the ask). So bid_price gets a smaller weight (ask_qty is small) and ask_price gets a larger weight (bid_qty is large), pulling the result toward the ask. This is a common source of confusion — the cross-weighting is intentional.

Worked Example

Suppose: bid = 100 @ qty 200, ask = 102 @ qty 50.

```
mkt_price = (100 * 50 + 102 * 200) / (200 + 50)
          = (5000 + 20400) / 250
          = 25400 / 250
          = 101.6
```

The fair price is 101.6, pulled toward 102 (the ask), because the bid side has much more depth — signaling buying pressure.

This feature updates on every **order book update**, inside the `onOrderBookUpdate()` callback. It is used by the **MarketMaker** strategy.

Feature 2: agg_trade_qty_ratio_ (Aggressive Trade Ratio)

```
agg_trade_qty_ratio_ = trade_qty / (aggressor_is_buyer ? ask_qty : bid_qty)
```

This measures how aggressive a trade was relative to the available liquidity. If someone sweeps 80% of the BBO quantity in a single trade, the ratio is 0.8 — strong trade pressure in that direction.

This feature updates on every **trade event**, inside the `onTradeUpdate()` callback. It is used by the **LiquidityTaker** strategy.

Design Detail: NaN Sentinel Value

```
constexpr auto Feature_INVALID = std::numeric_limits<double>::quiet_NaN();
```

Features start as NaN (not-a-number). Before the first relevant market event arrives, features are invalid. Strategies **must** check `!= Feature_INVALID` before using any feature value. Using NaN is safer than using 0 or -1, which are legitimate values for doubles.

Complete Code

onOrderBookUpdate — updates fair price

```
auto onOrderBookUpdate(TickerId ticker_id, Price price,
    Side side, MarketOrderBook* book) noexcept -> void {
  const auto bbo = book->getBBO();
  if(LIKELY(bbo->bid_price_ != Price_INVALID
           && bbo->ask_price_ != Price_INVALID)) {
    mkt_price_ = (bbo->bid_price_ * bbo->ask_qty_ +
                  bbo->ask_price_ * bbo->bid_qty_)
                 / static_cast<double>(
                   bbo->bid_qty_ + bbo->ask_qty_);
  }
}
```

onTradeUpdate — updates aggressive trade ratio

```
auto onTradeUpdate(const Exchange::MEMarketUpdate
    *market_update, MarketOrderBook* book) noexcept -> void {
  const auto bbo = book->getBBO();
  if(LIKELY(bbo->bid_price_ != Price_INVALID
           && bbo->ask_price_ != Price_INVALID)) {
    agg_trade_qty_ratio_ = static_cast<double>
      (market_update->qty_) / (market_update->side_ ==
       Side::BUY ? bbo->ask_qty_ : bbo->bid_qty_);
  }
}
```

> **LIKELY() macro:** A wrapper around __builtin_expect. Tells the compiler that this branch is the common case (prices are almost always valid). This allows the CPU's instruction cache and branch predictor to optimize the hot path. A standard HFT optimization.

02 — PositionKeeper (Position & PnL Tracking)

This component answers: what is my current position, how much have I made, and how much have I lost?

The core difficulty is correctly distinguishing realized PnL from unrealized PnL:

Realized PnL: Locked-in profit or loss. Occurs when you match a buy with a sell (or vice versa) — a position is partially or fully closed. Only changes when new executions reduce or close a position.

Unrealized PnL: Floating P&L on open positions. Even without new executions, this changes whenever market prices move. Represents what you would make or lose if you closed the position right now.

Total PnL = realized + unrealized

Data Structure: PositionInfo

```
struct PositionInfo {
    int32_t position_ = 0;            // net position: +long, -short, 0 flat
    double real_pnl_ = 0;             // realized PnL (closed positions)
    double unreal_pnl_ = 0;           // unrealized PnL (open position)
    double total_pnl_ = 0;            // real + unreal
    std::array<double,
      sideToIndex(Side::MAX)+1>
      open_vwap_;                     // BUY side and SELL side: sum(price * qty)
    Qty volume_ = 0;                  // total executed volume
    const BBO *bbo_ = nullptr;        // pointer to latest BBO
};
```

> **Key insight: open_vwap_ is misleadingly named.** It stores Σ(price × qty), NOT the VWAP itself. To get actual VWAP, divide by |position|: `VWAP = open_vwap_[side] / abs(position_)`. Why not store VWAP directly? Because accumulating the price×qty sum and dividing once is numerically exact. Incremental VWAP updates on floating-point numbers cause precision drift over many trades.

PnL Walkthrough: Full Numerical Example

This is the most important section. We trace 7 executions and watch position, VWAP, and PnL evolve. Starting from a completely flat position.

Step 1: Buy 10 @ 100 — Open a Long Position

```
position: 0 -> 10
open_vwap[BUY]: 0 -> 10 * 100 = 1000
VWAP = 1000 / 10 = 100
realized PnL: 0 (no position was closed)
unrealized PnL: 0 (just executed, price = entry price)
```

Simple: we opened a long. No PnL yet.

Step 2: Buy 10 @ 90 — Increase the Long

```
position: 10 -> 20
open_vwap[BUY]: 1000 -> 1000 + 10 * 90 = 1900
VWAP = 1900 / 20 = 95
realized PnL: 0 (still no matched pairs)
unrealized PnL: (90 - 95) * 20 = -100 (marked to BBO mid price)
```

We're down 100 because our average entry is 95 but the latest price is 90. Realized PnL is still 0 because we haven't sold anything — there's no buy-sell pair to match.

Step 3: Sell 10 @ 92 — Partial Close (Realized PnL Appears)

```
position: 20 -> 10
VWAP[BUY] remains 95 (open_vwap rescaled: 1900 → 95 * 10 = 950)
realized PnL: (92 - 95) * 10 = -30 (sold below buy VWAP)
unrealized PnL: (92 - 95) * 10 = -30 (remaining 10 shares still underwater)
```

This is the key moment. We sold 10 shares at 92 against a buy VWAP of 95. The difference of -3 per share × 10 shares = **-30 realized PnL**. The remaining 10 shares are still valued at the same VWAP of 95 against the latest price of 92, giving -30 unrealized.

Step 4: Sell 20 @ 97 — Position Flip (Long → Short)

This is the most complex step. We're long 10, sell 20, and flip to short 10. The system must **split this into two logical parts**:

```
position: 10 -> -10

Part 1: Close the long (first 10 shares):
  realized PnL += (97 - 95) * 10 = +20
  cumulative realized: -30 + 20 = -10

Part 2: Open a new short (remaining 10 shares):
  open_vwap[BUY] -> reset to 0
  open_vwap[SELL] -> 97 * 10 = 970
```

```
   unrealized PnL: 0 (VWAP = 97 = current price)
```

We sold 10 at 97 to close our long (VWAP 95), earning +20. The excess 10 shares open a new short position at 97. Since the VWAP equals the execution price, unrealized PnL starts at 0. Note how `open_vwap_[BUY]` is reset to 0 and `open_vwap_[SELL]` is initialized fresh.

Steps 5–6: Sell 20 @ 94, then Sell 10 @ 90 — Increase Short

```
Step 5: position: -10 -> -30
  open_vwap[SELL]: 970 + 20*94 = 2850, VWAP = 2850/30 = 95
  realized: -10 (unchanged, position increased)
  unrealized: (95 - 94) * 30 = 30 (short VWAP > market price = profit)

Step 6: position: -30 -> -40
  open_vwap[SELL]: 2850 + 10*90 = 3750, VWAP = 3750/40 = 93.75
  realized: -10 (unchanged)
  unrealized: (93.75 - 90) * 40 = 150
```

Symmetric to steps 1–2 but in the short direction. The short position increases, VWAP updates, unrealized grows because we shorted at higher prices than the current market.

Step 7: Buy 40 @ 88 — Flatten Completely

```
position: -40 -> 0 (flat)
realized += (93.75 - 88) * 40 = 230
total realized PnL: -10 + 230 = 220
unrealized PnL: 0 (flat)
all open_vwap reset to 0
```

We bought back our entire short at 88, against a short VWAP of 93.75. Profit: 5.75 per share × 40 = 230. Combined with the earlier -10, **final realized PnL = 220**. Position is flat, unrealized is 0.

Complete Summary Table

pos old	pos new	vwap BUY	vwap SELL	VWAP B	VWAP S	PnL real	PnL unrl
0	10	1000	0	100	0	0	0
10	20	1900	0	95	0	0	-100
20	10	950	0	95	0	-30	-30
10	-10	0	970	0	97	-10	0
-10	-30	0	2850	0	95	-10	30
-30	-40	0	3750	0	93.75	-10	150
-40	0	0	0	0	0	220	0

The addFill() Algorithm

The core logic that handles all four cases (increase, decrease, flip, flatten) in one method:

Increase vs. Decrease Detection

```
const auto old_position = position_;
position_ += client_response->exec_qty_ * sideToValue(client_response->side_);

if (old_position * sideToValue(client_response->side_) >= 0) {
    // Same direction as existing position -> INCREASE
    // Just accumulate into open_vwap
    open_vwap_[side_index] += price * exec_qty;
} else {
    // Opposite direction -> DECREASE (compute realized PnL)
    const auto opp_side_vwap = open_vwap_[opp_side_index] / abs(old_position);
    open_vwap_[opp_side_index] = opp_side_vwap * std::abs(position_);
    real_pnl_ += min(exec_qty, abs(old_position)) *
                 (opp_side_vwap - price) * sideToValue(side);
}
```

`sideToValue(BUY) = +1, sideToValue(SELL) = -1`. If old position is positive (long) and we receive a BUY fill, the product is positive — same direction, increase. If old position is positive but we receive a SELL fill, product is negative — opposite direction, decrease.

Notice how `sideToValue()` eliminates branches: `position_ += qty * sideToValue(side)` handles both BUY and SELL without any if/else. Eliminating branches is a standard HFT optimization because mispredicted branches cause pipeline stalls.

Position Flip Detection

```
if (position_ * old_position < 0)  // sign changed -> flipped
```

If the new and old positions have different signs, we flipped from long to short or vice versa. In this case, reset the opposite side's vwap to 0 and initialize the new side's vwap from the execution price.

Mark-to-Market on BBO Changes

```
auto mid_price = (bbo->bid_price_ + bbo->ask_price_) * 0.5;
if (position_ > 0)
    unreal_pnl_ = (mid_price - open_vwap_[BUY] / abs(position_)) * abs(position_);
else
    unreal_pnl_ = (open_vwap_[SELL] / abs(position_) - mid_price) * abs(position_);
```

Even without new executions, unrealized PnL must update when market prices move. If you have a long position and mid-price goes up, your unrealized profit increases. This method is called from PositionKeeper whenever the BBO changes.

PositionKeeper: Just a Routing Layer

```
class PositionKeeper {
```

```
    std::array<PositionInfo, ME_MAX_TICKERS> ticker_position_;
};
```

A fixed-size array indexed by TickerId. Routes `addFill()` and `updateBBO()` to the correct PositionInfo. O(1) lookup, zero heap allocation.

03 — OrderManager (Order Management)

The strategy says "I want a bid at 100 and an ask at 102." OrderManager figures out the mechanics: should it send a new order? Cancel an existing one? Wait for the exchange? It handles all of this.

OMOrder: The Order Object and State Machine

```
struct OMOrder {
    TickerId ticker_id_;    OrderId order_id_;
    Side side_;             Price price_;
    Qty qty_;               OMOrderState order_state_;
};

enum class OMOrderState : int8_t {
    INVALID = 0, PENDING_NEW = 1, LIVE = 2, PENDING_CANCEL = 3, DEAD = 4
};
```

The state transitions form a simple lifecycle:

```
DEAD/INVALID --newOrder()--> PENDING_NEW --ACCEPTED--> LIVE
                                                        |
                                                  cancelOrder()
                                                        |
                                                        v
DEAD <------CANCELED------- PENDING_CANCEL

LIVE --FILLED(leaves_qty=0)--> DEAD
```

> **Critical constraint: nothing can happen in PENDING states.** When an order is PENDING_NEW or PENDING_CANCEL, the system must wait for the exchange to respond before taking any action. This prevents state inconsistency from operating on unconfirmed orders.

Data Structure: One Order Per Side Per Ticker

```
typedef std::array<OMOrder, sideToIndex(Side::MAX) + 1> OMOrderSideHashMap;
typedef std::array<OMOrderSideHashMap, ME_MAX_TICKERS>  OMOrderTickerSideHashMap;
```

Major simplification: each instrument can have at most one buy order and one sell order. No order lists, no dynamic allocation. A fixed-size array indexed by TickerId and sideToIndex(). O(1) access, contiguous memory, cache-friendly.

moveOrders() — The Only Interface Strategies Need

```
auto moveOrders(TickerId tid, Price bid_price, Price ask_price, Qty clip) {
    auto bid_order = &(ticker_side_order_[tid][sideToIndex(Side::BUY)]);
```

```
    moveOrder(bid_order, tid, bid_price, Side::BUY, clip);

    auto ask_order = &(ticker_side_order_[tid][sideToIndex(Side::SELL)]);
    moveOrder(ask_order, tid, ask_price, Side::SELL, clip);
}
```

Strategy says: "bid at 100, ask at 102, 50 lots each." OrderManager handles the rest.

moveOrder() — Per-Side Decision Logic

```
switch (order->order_state_) {
    case LIVE:
        if (order->price_ != price || order->qty_ != qty)
            cancelOrder(order);  // price changed -> cancel, will re-place next cycle
        break;                   // price same -> do nothing

    case INVALID:
    case DEAD:
        if (price != Price_INVALID) {
            if (risk_manager_.checkPreTradeRisk(tid, side, qty) == ALLOWED)
                newOrder(order, tid, price, side, qty);
        }
        break;

    case PENDING_NEW:
    case PENDING_CANCEL:
        break;  // waiting for exchange, do nothing
}
```

Cancel-then-replace, not atomic modify: after cancelling, the system does NOT immediately re-send. It waits until the next moveOrders() call (triggered by the next market data update), when the order state will be DEAD and a new order can be placed. There's a brief window with no order on that side. Simple design, small tradeoff.

The Price_INVALID Trick

Passing `Price_INVALID` for one side causes that order to be cancelled (any LIVE order's price won't match Price_INVALID, triggering cancel) and no new order to be sent (the `price != Price_INVALID` check fails in the DEAD branch). LiquidityTaker uses this to only maintain an order on one side at a time.

To be concrete, here is what happens inside moveOrder() when Price_INVALID is passed:

```
// Case 1: order is LIVE, price = Price_INVALID
//   order->price_ != Price_INVALID  → true  → cancelOrder()
//
// Case 2: order is DEAD, price = Price_INVALID
//   price != Price_INVALID  → false  → skip newOrder()
//
// Net effect: existing order cancelled, no replacement sent.
```

```
// One sentinel value, two effects, zero extra branches.
```

Handling Exchange Responses: onOrderUpdate()

```
case ACCEPTED:  order->order_state_ = LIVE;   break;
case CANCELED:  order->order_state_ = DEAD;   break;
case FILLED:
    order->qty_ = client_response->leaves_qty_;
    if (!order->qty_) order->order_state_ = DEAD;  // fully filled
    break;
```

Note: FILLED updates the remaining quantity. If leaves_qty is still non-zero, the order stays LIVE (partial fill). Only fully filled orders become DEAD.

04 — RiskManager (Risk Controls)

RiskManager is the gatekeeper. Before every new order is sent, it must pass three checks. If any check fails, the order is rejected — it never reaches the exchange.

Risk Configuration

```
struct RiskCfg {
    Qty max_order_size_ = 0;   // max single order size (fat-finger protection)
    Qty max_position_  = 0;   // max absolute position
    double max_loss_   = 0;   // max allowed loss (negative, e.g. -100)
};
```

Two-Layer Architecture

RiskManager has a two-layer design that can be confusing at first glance. The outer layer (RiskManager class) has a 3-parameter method that routes by ticker; the inner layer (RiskInfo struct) has a 2-parameter method that performs the actual checks:

```
// Outer layer: RiskManager (3 params, routes by ticker)
auto checkPreTradeRisk(TickerId tid, Side side, Qty qty) {
    return ticker_risk_.at(tid).checkPreTradeRisk(side, qty);
}

// Inner layer: RiskInfo (2 params, does actual checks)
// (shown below)
```

OrderManager calls the 3-parameter version. It routes to the correct RiskInfo by TickerId, which then calls the 2-parameter version shown below. This is not a signature mismatch — it is a deliberate routing pattern, consistent with the “array-as-HashMap” design used throughout the system.

Three Checks, Sequential Execution

```
auto checkPreTradeRisk(Side side, Qty qty) const noexcept {

    // Check 1: Is this individual order too large?
    if (qty > risk_cfg_.max_order_size_)
        return ORDER_TOO_LARGE;

    // Check 2: Would the resulting position exceed limits?
    if (abs(position_ + qty * sideToValue(side)) > max_position_)
        return POSITION_TOO_LARGE;

    // Check 3: Have we already lost too much?
```

```
    if (total_pnl_ < max_loss_)
        return LOSS_TOO_LARGE;

    return ALLOWED;
}
```

Check 1 — ORDER_TOO_LARGE: Fat-finger protection. If max_order_size = 150 and you try to send 200, rejected immediately.

Check 2 — POSITION_TOO_LARGE: This is forward-looking. It doesn't check if the current position exceeds the limit — it checks what the position **would be** if this order were fully filled. Example: currently long 250, max = 300, want to buy 60 more: |250 + 60| = 310 > 300 → rejected. But sell 60: |250 - 60| = 190 < 300 → allowed. The reducing direction is always permitted.

Check 3 — LOSS_TOO_LARGE: Circuit breaker. Uses total_pnl (realized + unrealized). Once losses exceed the threshold, ALL new orders are blocked, regardless of direction.

> **Caveat:** This implementation doesn't distinguish opening from closing trades. If you hit the loss limit, you can't even send orders to close your losing position. In production, you'd want to still allow position-reducing orders when the loss limit is hit.

The Complete Call Chain: Four Components Wired Together

```
Strategy: "I want to buy 50 at 100"
  -> OrderManager::moveOrders()
    -> moveOrder() sees state = DEAD
      -> RiskManager::checkPreTradeRisk(BUY, 50)
        -> reads PositionKeeper's position_ and total_pnl_
        -> all three checks pass -> ALLOWED
      -> newOrder() -> constructs MEClientRequest{NEW, BUY, 100, 50}
        -> TradeEngine::sendClientRequest()
          -> writes to outgoing_ogw_requests_ LFQueue
            -> OrderGateway thread reads it -> TCP to exchange
```

This is the full hot path from trading signal to order. Everything within the TradeEngine thread is synchronous, lock-free, with zero heap allocations.

05 — Trading Strategies

Now we have all the building blocks. The strategy layer simply decides: under what conditions, at what prices, do I call moveOrders()?

The two strategies have identical structure but completely opposite logic.

MarketMaker (Passive Liquidity Provider)

Goal: Capture the spread. Post passive bids and asks on both sides. Buy at 100, sell at 102, both sides fill = profit of 2. Risk: adverse selection — you buy and the price keeps falling, or you sell and it keeps rising.

Decision trigger: `onOrderBookUpdate()` only. Does nothing on trade events. Market makers care about the static state of the order book, not individual trades.

The Complete Decision Logic

```
auto onOrderBookUpdate(TickerId ticker_id, Price price,
    Side side, const MarketOrderBook *book) noexcept -> void {
  const auto bbo = book->getBBO();
  const auto fair_price = feature_engine_->getMktPrice();

  if (LIKELY(bbo->bid_price_ != Price_INVALID &&
             bbo->ask_price_ != Price_INVALID &&
             fair_price != Feature_INVALID)) {

    const auto clip = ticker_cfg_.at(ticker_id).clip_;
    const auto threshold = ticker_cfg_.at(ticker_id).threshold_;

    const auto bid_price = bbo->bid_price_
        - (fair_price - bbo->bid_price_ >= threshold ? 0 : 1);
    const auto ask_price = bbo->ask_price_
        + (bbo->ask_price_ - fair_price >= threshold ? 0 : 1);

    order_manager_->moveOrders(ticker_id, bid_price, ask_price, clip);
  }
}
```

This is the **entire strategy**. The onTradeUpdate and onOrderUpdate methods just log and forward.

Pricing Logic Explained

Bid side: `bid = BBO.bid - (fair_price - BBO.bid >= threshold ? 0 : 1)`

If fair_price - bid >= threshold: fair price is well above the bid (buying at bid looks favorable) → subtract 0, join best bid (aggressive).

If fair_price - bid < threshold: fair price is too close to the bid (risky to buy here) → subtract 1, step back one tick (conservative).

Ask side: perfectly symmetric.

Worked Example

BBO: bid = 100 @ qty 500, ask = 102 @ qty 200, threshold = 0.6

```
fair_price = (100 * 200 + 102 * 500) / 700 = 101.43

Bid: 101.43 - 100 = 1.43 >= 0.6 -> join at 100 (aggressive)
Ask: 102 - 101.43 = 0.57 < 0.6  -> step back to 103 (conservative)
```

Fair price leans toward the ask — buying pressure is strong. The strategy is aggressive on the bid (willing to buy at the best price) but conservative on the ask (stepping back because price may be heading up).

LiquidityTaker (Aggressive Directional Trader)

Goal: Follow large trades. When someone slams a huge sell order, possibly an informed trader dumping, the LiquidityTaker follows by selling aggressively too. Vice versa for buys.

Decision trigger: `onTradeUpdate()` only. Does nothing on book updates. Exact opposite of MarketMaker.

The Complete Decision Logic

```
auto onTradeUpdate(const MEMarketUpdate *market_update,
    MarketOrderBook *book) noexcept -> void {
  const auto bbo = book->getBBO();
  const auto agg_qty_ratio = feature_engine_->getAggTradeQtyRatio();

  if (LIKELY(bbo valid && agg_qty_ratio != Feature_INVALID)) {
    const auto clip = ticker_cfg_.at(ticker_id).clip_;
    const auto threshold = ticker_cfg_.at(ticker_id).threshold_;

    if (agg_qty_ratio >= threshold) {
      if (market_update->side_ == Side::BUY)
        order_manager_->moveOrders(tid, bbo->ask_price_, Price_INVALID, clip);
      else
        order_manager_->moveOrders(tid, Price_INVALID, bbo->bid_price_, clip);
    }
  }
}
```

Only one side at a time. Buy aggressor → send buy at ask_price, pass Price_INVALID for sell side. Sell aggressor → send sell at bid_price, pass Price_INVALID for buy side.

MarketMaker vs LiquidityTaker — Side by Side

Dimension	MarketMaker	LiquidityTaker
Profit source	Capture spread	Directional prediction
Order type	Passive (rest in book)	Aggressive (cross spread)
Decision trigger	onOrderBookUpdate()	onTradeUpdate()
Feature used	mkt_price_ (fair price)	agg_trade_qty_ratio_
Order sides	Both bid + ask always	One side at a time
Inventory risk	High (always has position)	Lower (trade then done)
Spread impact	Earns the spread	Pays the spread
Order mgmt	Continuous adjustment	Fire-and-forget

They naturally trade against each other. In the book's ecosystem: 2 MarketMakers provide liquidity, 2 LiquidityTakers consume it, 1 Random strategy generates background noise. A minimal but complete market microstructure.

06 — TradeEngine (End-to-End Integration)

TradeEngine is the hub. It doesn't make trading decisions. It moves data in from external threads, distributes it to internal components, and moves orders out.

The run() Loop: Busy Polling

```
auto TradeEngine::run() noexcept -> void {
    while (run_) {
        // 1. Drain order responses
        for (auto resp = incoming_ogw_responses_->getNextToRead();
             resp; resp = incoming_ogw_responses_->getNextToRead()) {
            onOrderUpdate(resp);
            incoming_ogw_responses_->updateReadIndex();
        }
        // 2. Drain market data updates
        for (auto update = incoming_md_updates_->getNextToRead();
             update; update = incoming_md_updates_->getNextToRead()) {
            ticker_order_book_[update->ticker_id_]->onMarketUpdate(update);
            incoming_md_updates_->updateReadIndex();
        }
    }
}
```

No sleep. No select. No epoll. Pure busy polling. The thread spins forever, processing data the instant it appears. This is standard HFT practice: trade one CPU core for minimum latency.

Three Data Paths

Path 1: Market Data In → Strategy Decision

```
MarketDataConsumer thread:
  UDP multicast -> decode -> write to incoming_md_updates_ LFQueue

TradeEngine thread (run loop reads it):
  -> ticker_order_book_[tid]->onMarketUpdate()
    -> order book updates BBO
    -> callback: TradeEngine::onOrderBookUpdate()
      -> position_keeper_.updateBBO()          // update unrealized PnL
      -> feature_engine_.onOrderBookUpdate()    // update mkt_price_
      -> algoOnOrderBookUpdate_()               // forward to strategy
        -> MarketMaker::onOrderBookUpdate()     // strategy decides
          -> order_manager_->moveOrders()       // send/cancel orders
```

Path 2: Order Request Out

```
Strategy calls order_manager_->moveOrders()
```

```
  -> risk_manager_.checkPreTradeRisk() -> ALLOWED
  -> newOrder() -> MEClientRequest
  -> trade_engine_->sendClientRequest()
    -> write to outgoing_ogw_requests_ LFQueue

OrderGateway thread:
  -> read from LFQueue -> TCP send to exchange
```

Path 3: Order Response Back

```
OrderGateway thread:
  -> receive from TCP -> write to incoming_ogw_responses_ LFQueue

TradeEngine thread (run loop reads it):
  -> TradeEngine::onOrderUpdate()
    -> if FILLED: position_keeper_.addFill()  // update position & PnL
    -> algoOnOrderUpdate_()                   // forward to strategy
      -> order_manager_->onOrderUpdate()      // update OMOrder state
```

Callback Wiring via std::function

```
// In MarketMaker constructor:
trade_engine->algoOnOrderBookUpdate_ =
    [this](auto tid, auto price, auto side, auto book) {
        onOrderBookUpdate(tid, price, side, book);
    };
```

TradeEngine doesn't know which strategy it's running. It invokes the callback; the lambda routes the call. Clean decoupling — adding a new strategy type doesn't require modifying TradeEngine.

Complete Order Lifecycle: 10 Steps

```
 1. [TradeEngine]          Market data arrives, BBO changes
 2. [TradeEngine]          MarketMaker::onOrderBookUpdate()
                           fair_price=101.43 -> bid=100, ask=103
 3. [TradeEngine]          order_manager_->moveOrders(tid, 100, 103, 50)
 4. [TradeEngine]          bid OMOrder is DEAD -> risk check ALLOWED
                           -> newOrder() -> write to LFQueue
                           -> state = PENDING_NEW
 5. [OrderGateway thread]  reads request -> TCP to exchange
 6. [Exchange]             MatchingEngine -> ACCEPTED + ADD market update
 7. [OrderGateway thread]  receives ACCEPTED -> writes to incoming LFQueue
 8. [TradeEngine]          reads response -> state = LIVE
 9. [MarketDataConsumer]   receives ADD update -> writes to md LFQueue
10. [TradeEngine]          reads update -> book updates -> strategy re-evaluates
```

On a tuned system, the entire path from market data arrival to order hitting the wire is on the order of microseconds.